Spis treści

1. Specyfikacja projektu 2
2. Opis czujnika MPU-6050 2
3. Projekt Protokołu Komunikacyjnego 3
3.1. Struktura Ramki 4
3.2. Opis Pól Ramki 4
3.3. Mechanizm Byte-Stuffing 5
4. Komendy Systemowe 5
5. Szczegółowy Opis Komend i Odpowiedzi 6
5.1. Komenda CMD_START (0x10) 6
5.2. Komenda CMD_STOP (0x11) 7
5.3. Komenda CMD_GET_DATA (0x12) 8
5.4. Komenda CMD_GET_BUFFER (0x13) 9
5.5. Komenda CMD_SET_TIME (0x14) 10
5.6. Komenda CMD_GET_ARCHIVE (0x15) 11
5.7. Szczegółowy opis danych pomiarowych (MPU6050) 12
6. Obsługa Błędów 12
7. Implementacja Programowa 13
7.1. Maszyna Stanów (Parser Ramki) 13
7.2. Funkcje Nieblokujące (DMA) 14
7.3. Suma CRC-32 14
8. Konfiguracja pinów i liczników 16
8.1. Tabela wyprowadzeń (Pinout) 16
8.2. Konfiguracja interfejsów komunikacyjnych 17
8.3. Konfiguracja Timera systemowego 17
8.4. Konfiguracja interfejsu USART2 (Komunikacja z PC) 18
8.5. Konfiguracja magistrali I2C1 z obsługą DMA 18
8.6. Mechanizm buforowania (Ring Buffer) 19

Specyfikacja projektu

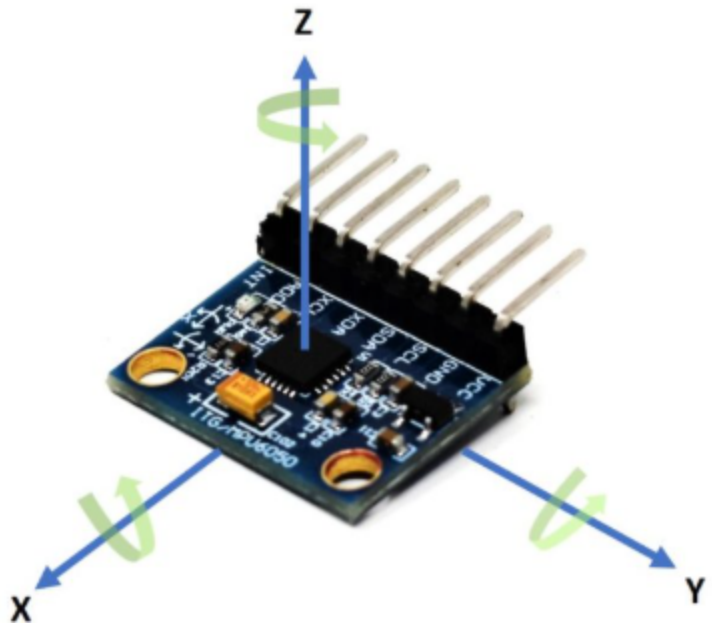
Projekt bedzie zrealizowany na płytce STM32 Nucleo-F401RE (seria Nucleo-64) i polega na zaprogramowaniu mikrokontrolera STM32 do wspolpracy z Akcelerometrem 3-Osiowym GY521 MPU6050, który wykrywa ruch lub wstrząsy płytki i sygnalizuje je zapaleniem diody LED. Informacje o wykryciu ruchu oraz dane pomiarowe z czujnika są przesyłane do komputera przez interfejs UART z wykorzystaniem przerwań, buforów kołowych i prostego protokołu komunikacyjnego zapewniającego poprawność transmisji danych.

1. Oprogramować komunikacje mikroprocesora z PC z wykorzystaniem przerwań i buforów kołowych poprzez interfejs asynchroniczny.
2. Zaprojektować i zaimplementować protokół komunikacyjny pozwalający na adresowanie pakietów, przekazywanie dowolnych danych, weryfikację poprawności przesłanych danych z uwzględnieniem ich kolejności.
3. Obsłużyć czujnik podłączony magistralą I²C z wykorzystaniem funkcji nieblokujących (DMA). Odczytywać dane periodycznie co zadany interwał czasowy w ms(uint32_t). Odczytane próbki zapisywać do bufora cyklicznego (circular buffer) o pojemności 1024 pozycji. Bufor ma przechowywać zarówno dane bieżące (najświeższe), jak i archiwalne. System powinien umożliwiać z PC podgląd do bufora

2. Opis czujnika MPU-6050

Czujnik MPU6050 (stosowany w module GY-521) jest układem typu MEMS przeznaczonym do pomiaru przyspieszeń w trzech osiach: X, Y, Z w programowalnym zakresie , , lub . Pomiar realizowany jest na zasadzie detekcji przemieszczeń mikroskopijnych mas wewnątrz struktury krzemowej czujnika, których odchylenie pod wpływem siły bezwładności jest przekształcane na sygnał elektryczny, a następnie na wartość cyfrową przez 16-bitowe przetworniki ADC. Na podstawie tych pomiarów wyznaczany jest wektor przyspieszenia działający na czujnik. Do komunikacji z jednostką centralną wykorzystywana jest magistrala I2C, umożliwiająca łatwą integrację czujnika z mikrokontrolerem.

Orientacja osi pomiarowych X, Y, Z trójosiowego akcelerometru, definiujących kierunki, w których czujnik mierzy wektory przyspieszenia:



Czujnik MPU6050 posiada wewnętrzne rejestry konfiguracyjne oraz rejestry danych, które umożliwiają sterowanie jego pracą i odczyt wyników pomiarów. Rejestr `WHO_AM_I` (adres `0x75`) służy do identyfikacji układu i sprawdzenia poprawności komunikacji z czujnikiem (oczekiwana wartość `0x68`). Rejestr `PWR_MGMT_1` jest kluczowy do wybudzenia układu z domyślnego stanu uśpienia. Rejestry `ACCEL_CONFIG` pozwalają na konfigurację zakresu pomiarowego (czułości). Wyniki pomiarów przyspieszeń w osiach X, Y, Z zapisywane są w parach rejestrów (np. `ACCEL_XOUT_H` i `ACCEL_XOUT_L`).

Podłączenie czujnika (moduł GY-521) do STM32 Nucleo-64:

Pin modułu GY-521	Do czego w STM32
VCC	5V lub 3.3V
GND	GND
SCL	PB8 (I2C1 SCL)
SDA	PB9 (I2C1 SDA)

3. Projekt Protokołu Komunikacyjnego

Protokół został zaprojektowany jako **binarny**, co oznacza, że wszystkie wartości liczbowe (pomiary, liczniki, sumy kontrolne) przesyłane są jako surowe bajty (binarnie), a nie w postaci tekstowej ASCII. Zapewnia to wyższą wydajność i mniejszy narzut danych.

3.1 Struktura Ramki

Ramka danych posiada stały nagłówek i stopkę, a jej zawartość jest zabezpieczona przed błędną
interpretacją znaków sterujących.

Pole	Początek (SOF)	SEQ	Nadawca (SRC)	Odbiorca (DST)	Komenda (CMD)	Dane	CRC-32	Koni (EOF
Rozmiar	1 B	1 B	1 B	1 B	1 B	B	4 B	1 B
Opis	`0x7E`	Licznik	ID	ID	Kod	Payload	Suma	`0x7C`

Minimalna długość ramki (bez danych): 10 bajtów.
`SOF(1) + SEQ(1) + SRC(1) + DST(1) + CMD(1) + CRC(4) + EOF(1) = 10`

Maksymalna długość ramki:

- Maksymalna długość logiczna ramki wynosi **265 bajtów** (10 bajtów stałych pól nagłówka i stopki oraz maksymalnie 255 bajtów pola danych).
- Rzeczywista długość ramki przesyłanej przez łącze fizyczne powiększona jest o ewentualne bajty dodane przez mechanizm **byte-stuffing** (znaki ucieczki `0x7D` wstawiane przed bajtami specjalnymi).

Reprezentacja danych:

- Wszystkie bajty w dokumentacji reprezentowane są w notacji heksadecymalnej (np. `0x7E`).
- Dane przesyłane są magistralą w postaci binarnej, w jednostkach 8-bitowych.
- Wartości wielobajtowe (np. 16-bitowe pomiary z akcelerometru, 32-bitowa suma CRC) przesyłane są w konwencji **Little-Endian** (najpierw młodszy bajt).
- Liczby ze znakiem (pomiary przyspieszenia) reprezentowane są w kodzie uzupełnień do dwóch (U2).

3.2 Opis Pól Ramki

- **SOF (`0x7E`)**: Bajt startu (*Start Of Frame*). Resetuje maszynę stanów odbiornika.
- **SEQ**: 1-bajtowy numer sekwencyjny (licznik modulo 256). Służy do wykrywania zgubionych ramek.
  - Nadajnik inkrementuje SEQ o 1 dla każdej nowej ramki.
  - Odbiornik sprawdza ciągłość: `SEQ_oczekiwany == (SEQ_odebrany - 1)`.
- **Nadawca / Odbiorca**: 1-bajtowe unikalne identyfikatory urządzeń.
  - `0x01`: Komputer PC (Master)

- `0x02`: Mikrokontroler STM32 (Slave)
- **Komenda**: 1-bajtowy kod operacji (opisany w sekcji 4). Umieszczenie komendy w nagłówku (przed danymi) ułatwia szybkie parsowanie.
- **Dane**: Blok danych o zmiennej długości (0–255 bajtów). Zawiera argumenty komend lub wyniki pomiarów (int16_t).
- **CRC-32**: Suma kontrolna obliczana dla pól: `SEQ`, `Nadawca`, `Odbiorca`, `Komenda`, `Dane`. Zapewnia integralność danych.
- **EOF (`0x7C`)**: Bajt końca (*End Of Frame*).

3.3 Mechanizm Byte-Stuffing

Ponieważ dane są binarne, wewnątrz ramki mogą wystąpić bajty identyczne jak znaki sterujące (`0x7E`, `0x7C`, `0x7D`). Aby tego uniknąć, stosuje się mechanizm "ucieczki" (escape) dla bajtów danych, CRC i nagłówka (poza SOF/EOF).
Obliczenie CRC-32 wykonywane jest na ramce logicznej przed zastosowaniem mechanizmu byte-stuffing.

Zasady ogólne:

- Mechanizm stuffingu stosowany jest dla wszystkich pól ramki znajdujących się **pomiędzy** znakami SOF i EOF (SEQ, Nadawca, Odbiorca, Komenda, Dane, CRC).
- Obliczenie sumy kontrolnej **CRC-32** wykonywane jest na danych oryginalnych (ramce logicznej) **przed** zastosowaniem mechanizmu byte-stuffing.

Znaki objęte stuffingiem:

Bajt oryginalny	Znaczenie	Sekwencja wyjściowa (po kodowaniu)
`0x7E`	SOF (Start of Frame)	`0x7D` `0x5E`
`0x7C`	EOF (End of Frame)	`0x7D` `0x5C`
`0x7D`	ESC (Escape Byte)	`0x7D` `0x5D`

Algorytm kodowania:

```
Bajt_zakodowany = 0x7D, (Bajt_oryginalny XOR 0x20)
```

4. Komendy Systemowe

Komendy identyfikowane są przez 1-bajtowy kod. Argumenty wielobajtowe (np. czas, wartości osi) przesyłane są w konwencji **Little-Endian** (najpierw młodszy bajt), chyba że zaznaczono inaczej.

Kod	Nazwa	Argumenty (Dane)	Odpowiedź STM32 (Dane)	Opis
`0x10`	**CMD_START**	`Interval` (4B)	`STATUS` (1B)	Uruchamia timer i cykliczny odczyt DMA. `Interval`.
`0x11`	**CMD_STOP**	Brak	`STATUS` (1B)	Zatrzymuje timer i akwizycję danych.
`0x12`	**CMD_GET_DATA**	Brak	`X, Y, Z` (6B)	Zwraca **najnowszą** próbkę z bufora.
`0x13`	**CMD_GET_BUFFER**	`Offset` (2B), `Count` (1B)	`N * (X,Y,Z)`	Zwraca `Count` próbek historycznych od indeksu `Offset`.
`0x14`	**CMD_SET_TIME**	`Time` (4B)	`STATUS` (1B)	Ustawia systemowy czas odniesienia (timestamp).
`0x15`	**CMD_GET_ARCHIVE**	`Back` (2B), `Count` (1B)	`N * (X,Y,Z)`	Zwraca `Count` próbek, cofając się o `Back` od głowy bufora.

5. Szczegółowy Opis Komend i Odpowiedzi

Wszystkie wartości wielobajtowe (argumenty komend, wyniki pomiarów) przesyłane są w konwencji **Little-Endian** (najmniej znaczący bajt jako pierwszy), chyba że zaznaczono inaczej.

Legenda do przykładów:

- **Adres PC (Master):** `0x01`
- **Adres STM32 (Slave):** `0x02`
- **CRC:** W przykładach oznaczono jako `CC CC CC CC` (należy obliczyć dla konkretnych danych).
- **Byte-stuffing:** W przykładach podano wartości "czyste" (przed stuffingiem) dla czytelności.

5.1 Komenda CMD_START (`0x10`)

Rozpoczyna cykliczne zbieranie danych z akcelerometru. Uruchamia Timer sprzętowy, który wyzwala transfer DMA z czujnika do bufora cyklicznego.

co jeśli `CMD_START` zostanie wysłane, gdy akwizycja już trwa?

- komenda ignorowana, zwracany STATUS błędu
- **Argumenty (1 Bajt):**

- `INTERVAL` (4B): Okres próbkowania podany bezpośrednio w milisekundach (typ `uint32_t`, Little-Endian).
- Zakres: `1` (10ms) - `255` (2550ms).

- **Odpowiedź:** Ramka statusu `ACK`.

Przykład:

Ustawienie próbkowania co **500 ms** (500 dec = `0x000001F4`). W formacie Little-Endian: `F4` `01` `00` `00`.

TX (PC STM):

SOF	SEQ	Nadawca	Odbiorca	CMD	T0	T1	T2	T3	CRC-32	EOF
7E	01	01	02	10	F4	01	00	00	CC CC CC CC	7C

RX (STM PC): Potwierdzenie przyjęcia (`STATUS = 0x00` OK).

SOF	SEQ	Nadawca	Odbiorca	CMD	DANE (Status)	CRC-32	EOF
`7E`	`01`	`02`	`01`	10	00	`CC CC CC CC`	`7C`

Odpowiedz STM:

```
TX [Cmd=0x10]: 7E 01 01 02 10 F4 01 00 00 E6 3D 55 DF 7C
RX [Raw]: 7E 01 02 01 10 00 5C C3 8E A3 7C
```

5.2 Komenda CMD_STOP (`0x11`)

Zatrzymuje Timer i akwizycję danych. Dane w buforze pozostają dostępne do odczytu.

- **Argumenty:** Brak (długość pola DANE = 0).
- **Odpowiedź:** Ramka statusu `ACK`.

Przykład:

TX (PC STM):

SOF	SEQ	Nadawca	Odbiorca	CMD	DANE	CRC-32	EOF
`7E`	`02`	`01`	`02`	`11`	`-`	`CC CC CC CC`	`7C`

RX (STM PC):

SOF	SEQ	Nadawca	Odbiorca	CMD	DANE (Status)	CRC-32	EOF
`7E`	`02`	`02`	`01`	`11`	`00`	`CC CC CC CC`	`7C`

Odpowiedz STM:

```
TX [Cmd=0x11]: 7E 01 01 02 11 83 B3 93 F2 7C
RX [Raw]: 7E 01 02 01 11 00 80 02 97 71 7C
```

5.3 Komenda CMD_GET_DATA (`0x12`)

Pobiera **jeden, najnowszy** kompletny pomiar (X, Y, Z) z bufora cyklicznego. Komenda nie modyfikuje wskaźników bufora.

- **Argumenty:** Brak
- **Odpowiedź:** 6 bajtów danych pomiarowych (`X, Y, Z`)

Format danych odpowiedzi (6 bajtów)

Dane są 16-bitowymi liczbami ze znakiem (`int16_t`), przesyłanymi w konwencji **Little-Endian**:

Kolejność	Pole
1	`ACCEL_X_L`
2	`ACCEL_X_H`
3	`ACCEL_Y_L`
4	`ACCEL_Y_H`
5	`ACCEL_Z_L`
6	`ACCEL_Z_H`

Przykład

Otrzymano pomiar:

- X = 250 (`0x00FA`)
- Y = −1 (`0xFFFF`)
- Z = 4000 (`0x0FA0`)

TX (PC STM):

SOF	SEQ	Nadawca	Odbiorca	CMD	DANE	CRC-32	EOF
`7E`	`03`	`01`	`02`	`12`	`-`	`CC CC CC CC`	`7C`

RX (STM PC):

SOF	SEQ	Nadawca	Odbiorca	CMD	X_L	X_H	Y_L	Y_H	Z_L	Z_H	CI 32
`7E`	`03`	`02`	`01`	`12`	`FA`	`00`	`FF`	`FF`	`A0`	`0F`	`C` `C` `C` `C`

Odpowiedz STM:

```
TX [Cmd=0x12]: 7E 02 01 02 12 34 27 A7 9F 7C
RX [Raw]: 7E 02 02 01 12 D0 FC 54 C3 E8 16 D4 1D 1F 83 7C
>> WYNIK: X=-816, Y=-15532, Z=5864
```

5.4 Komenda CMD_GET_BUFFER (`0x13`)

Pobiera historię pomiarów z bufora cyklicznego, zaczynając od wskazanego indeksu absolutnego (`Offset`). Służy do pobierania danych archiwalnych w blokach.

- **Argumenty (3 Bajty):**
  - `OFFSET` (2B): Indeks tablicy (0-1023), od którego ma rozpocząć się odczyt (Little-Endian).
  - `COUNT` (1B): Liczba próbek do pobrania (Maksimum: **42**, ze względu na limit długości ramki).
- **Odpowiedź:** Ciąg bajtów o długości `COUNT` × 6.

Przykład

Pobranie **2 próbek** zaczynając od indeksu **512** (`0x0200`).

- **Offset:** `512` (dec) = `0x0200` (hex). W Little-Endian: `00 02`.
- **Count:** `2` (dec) = `0x02` (hex).

Jeśli `OFFSET + COUNT` przekracza koniec bufora, STM32 zwraca dane z uwzględnieniem zawijania indeksu (modulo 1024).

TX (PC STM):**

SOF	SEQ	Nadawca	Odbiorca	CMD	OFF_L	OFF_H	COUNT	CRC-32	EOF
`7E`	`04`	`01`	`02`	`13`	`00`	`02`	`02`	`CC CC CC CC`	`7C`

RX (STM PC): *(Odbiór 12 bajtów danych: Próbka 1 [6B] + Próbka 2 [6B])*

SOF	SEQ	Nadawca	Odbiorca	CMD	DANE (Próbka 1)	DANE (Próbka 2)	CRC-32	EOF
`7E`	`04`	`02`	`01`	`13`	`XX XX XX XX XX XX`	`YY YY YY YY YY YY`	`CC CC CC CC`	`7C`

Odpowiedz STM:

```
TX [Cmd=0x13]: 7E 03 01 02 13 00 02 02 BB 89 DC E7 7C
RX [Raw]: 7E 03 02 01 13 C4 FC 4C C3 FC 16 20 FD 34 C3 14 17 8C 97 A7 F2 7C
>> DANE BUFORA (2): [(-828, -15540, 5884), (-736, -15564, 5908)]
```

5.5 Komenda CMD_SET_TIME (`0x14`)

Ustawia czas odniesienia (*timestamp*), który będzie kojarzony z kolejnymi próbkami zapisywanymi w buforze cyklicznym.

- **Argumenty (4 bajty):**
  - `TIME` (4B): Czas w milisekundach, typ `uint32_t` , przesyłany w konwencji **Little-Endian**
- **Odpowiedź:** Ramka statusu `ACK`

Przykład

Ustawienie czasu odniesienia na **100 000 ms** (`0x000186A0`).

TX (PC STM):

SOF	SEQ	Nadawca	Odbiorca	CMD	T0	T1	T2	T3	CRC-32	EOF
`7E`	`05`	`01`	`02`	`14`	`A0`	`86`	`01`	`00`	`CC CC CC CC`	`7C`

RX (STM PC): Potwierdzenie `STATUS = 0x00` .

SOF	SEQ	Nadawca	Odbiorca	CMD	DANE (Status)	CRC-32	EOF
`7E`	`05`	`02`	`01`	`14`	`00`	`CC CC CC CC`	`7C`

Odpowiedz STM:

```
TX [Cmd=0x14]: 7E 04 01 02 14 A0 86 01 00 0A 38 F0 AF 7C
RX [Raw]: 7E 04 02 01 14 00 FB 06 53 8F 7C
```

5.6 Komenda CMD_GET_ARCHIVE (`0x15`)

Pobiera dane wsteczne względem aktualnej pozycji zapisu (głowy bufora). Komenda umożliwia odczyt próbek zapisanych „przed chwilą”.

- **Argumenty (3 bajty):**
  - `BACK` (2B): O ile próbek cofnąć się od głowy bufora (Little-Endian)
  - `COUNT` (1B): Liczba próbek do pobrania (maks. **42**)
- **Odpowiedź:** Ciąg bajtów o długości `COUNT × 6`

Algorytm (STM32)

1. Oblicz indeks startowy:
   `START_IDX = (HEAD - BACK + 1024) % 1024`
2. Skopiuj `COUNT` próbek do bufora TX, obsługując zawijanie bufora (modulo 1024).

Przykład

Najnowsza próbka znajduje się pod indeksem **100**.
Pobieramy **5 ostatnich próbek** (`BACK = 5` , `COUNT = 5`).

- `BACK = 5` → `05 00` (Little-Endian)
- `COUNT = 5` → `05`

TX (PC STM):

SOF	SEQ	Nadawca	Odbiorca	CMD	BACK_L	BACK_H	COUNT	CRC-32	EOF
`7E`	`06`	`01`	`02`	`15`	`05`	`00`	`05`	`CC CC CC CC`	`7C`

RX (STM PC):

(Odbiór 30 bajtów danych: 5 próbek × 6 bajtów)

SOF	SEQ	Nadawca	Odbiorca	CMD	DANE (Próbki)	CRC-32	EOF
`7E`	`06`	`02`	`01`	`15`	`S1[6B] S2[6B] S3[6B] S4[6B] S5[6B]`	`CC CC CC CC`	`7C`

Odpowiedz STM:

```
TX [Cmd=0x15]: 7E 05 01 02 15 05 00 05 27 50 0E 38 7C
RX [Raw]: 7E 05 02 01 15 EC FC 04 C3 18 17 98 FD 28 C3 00 17 EC FC 24 C3 BC 16 CC FC 20 C3 F8 16 D0 FC 54 C3 E8 16 BC 2D B3 C3 7C
>> DANE ARCHIWUM (5): [(-788, -15612, 5912), (-616, -15576, 5888), (-788, -15580, 5820), (-820, -15584, 5880), (-816, -15532, 5864)]
```

Szczegółowy opis danych pomiarowych (MPU6050)

Czujnik MPU6050 zwraca przyspieszenie jako 16-bitowe liczby ze znakiem (`int16_t` , kod U2). W ramce `CMD_GET_DATA` (oraz innych komend zwracających pomiary) format danych (6 bajtów) wygląda następująco:

Wartości przyspieszenia rekonstruowane są w STM32/PC jako liczby typu `int16_t` (kod U2).

Offset (Bajt)	Nazwa pola	Typ danych	Opis
0	`ACCEL_X_L`	`uint8_t`	Młodszy bajt osi X (LSB)
1	`ACCEL_X_H`	`uint8_t`	Starszy bajt osi X (MSB)
2	`ACCEL_Y_L`	`uint8_t`	Młodszy bajt osi Y (LSB)
3	`ACCEL_Y_H`	`uint8_t`	Starszy bajt osi Y (MSB)
4	`ACCEL_Z_L`	`uint8_t`	Młodszy bajt osi Z (LSB)
5	`ACCEL_Z_H`	`uint8_t`	Starszy bajt osi Z (MSB)

> **Uwaga:** MPU6050 przechowuje dane w rejestrach w konwencji Big-Endian. STM32 przed wysłaniem danych do PC musi je przekonwertować do Little-Endian, zgodnie z przyjętą w projekcie konwencją.

6. Obsługa Błędów

Jeśli STM32 wykryje błąd w ramce zapytania, nie wykonuje komendy i zwraca ramkę błędu. Ramka błędu wysyłana jest przez STM32 do PC (Nadawca = `0x02` , Odbiorca = `0x01`).

Format Ramki Błędu:

- **Pole KOMENDA:** `0xFF` (Stały kod błędu).
- **Pole DANE:** 1 bajt kodu przyczyny (ERROR_CODE).

Kod (Hex)	Nazwa błędu	Opis przyczyny
0xE0	ERR_FRAME_LEN	Ramka zbyt krótka, brak nagłówka (SOF) lub stopki (EOF).
0xE1	ERR_CRC	Suma kontrolna obliczona przez STM32 nie zgadza się z odebraną.
0xE2	ERR_SEQ	Numer sekwencyjny nie jest kolejnym oczekiwanym (różnica większa niż 1, przy liczeniu modulo 256)
0xE3	ERR_UNKNOWN_CMD	Kod komendy nie istnieje w tablicy komend.
0xE4	ERR_ARG_RANGE	Argument komendy poza zakresem (np. żądanie 100 próbek, gdy limit to 42).
0xE5	ERR_BUFFER_FULL	Przepełnienie wewnętrznego bufora RX UART.

Przykład błędu CRC

PC wysyła uszkodzoną ramkę (np. zakłócenie na linii transmisyjnej powoduje zmianę bitu danych).

TX (PC STM): (Ramka z błędnym CRC)

SOF	SEQ	Nadawca	Odbiorca	CMD	DANE	CRC-32 (błędne)	EOF
7E	05	01	02	12	…	XX XX XX XX	7C

RX (STM PC) – ramka błędu :

SOF	SEQ	Nadawca	Odbiorca	CMD (Error)	DANE (ERROR_CODE)	CRC-32	EOF
7E	05	02	01	**FF**	**E1**	CC CC CC CC	7C

(Gdzie `E1` oznacza `ERR_CRC`)

7. Implementacja Programowa

7.1 Maszyna Stanów (Parser Ramki)

Odbiór danych w przerwaniu UART wrzuca bajty do bufora kołowego. W pętli głównej (`main`) dane są pobierane i analizowane przez maszynę stanów:

1. **RX_IDLE**: Czeka na bajt `0x7E`.
2. **RX_IN_FRAME**: Pobiera kolejne bajty aż do otrzymania znaku EOF (`0x7C`), obsługuje byte-stuffing (po odebraniu bajtu ESC `0x7D` pobiera kolejny bajt i wykonuje operację XOR `0x20`).
3. **STATE_CHECK**: Po otrzymaniu `0x7C` weryfikuje długość ramki oraz poprawność sumy CRC-32.
4. **STATE_EXEC**: Jeśli CRC poprawne → wykonuje funkcję obsługi przypisaną do odebranej komendy.

7.2 Funkcje Nieblokujące (DMA)

Odczyt z MPU6050 nie obciąża rdzenia CPU.

- Pętla główna programu (`while(1)`) w sposób nieblokujący sprawdza aktualny czas systemowy (funkcja `HAL_GetTick()`).
- Kiedy upłynie czas zdefiniowany komendą `CMD_START` (interwał w ms), główna pętla programu inicjuje nieblokujący odczyt funkcją `HAL_I2C_Mem_Read_DMA()`.

3. Magistrala I2C przesyła 6 bajtów do tymczasowej tablicy w pamięci RAM.
4. Po zakończeniu transferu, kontroler DMA wywołuje przerwanie `callback` `HAL_I2C_MemRxCpltCallback()`.
5. W funkcji callback DMA dane są kopiowane do Głównego Bufora Cyklicznego w pamięci RAM jako jedna próbka (X, Y, Z).
   Następnie aktualizowany jest wskaźnik HEAD bufora (pojemność: 1024 próbki).

7.3 Suma CRC-32

Parametr	Wartość
Algorytm	CRC-32 / MPEG-2
Init	0xFFFFFFFF
Wielomian	0x04C11DB7
ReflectedIn	False
ReflectedOut	False
XorOut	0x00000000
Zakres	SEQ + Nadawca + Odbiorca + Komenda + Dane
Przetwarzanie bitów	MSB-first

CRC-32 obliczane jest na **ramce logicznej**, przed zastosowaniem mechanizmu byte-stuffing.

- Pole **SEQ** umożliwia wykrywanie utraconych lub zdublowanych ramek.
- Mechanizm **byte-stuffing** zapewnia transparentność danych.
- **CRC-32** gwarantuje integralność przesyłanych danych.

Przykład obliczania CRC-32 (CRC-32/MPEG-2)

Dane wejściowe (2 bajty):

```
0x12 0x34
```

Dane są przetwarzane **bit po bicie**, od **najstarszego bitu (MSB)**.

1. Inicjalizacja rejestru CRC

`CRC = 0xFFFFFFFF`

2. Przetwarzanie pierwszego bajtu (0x12)

`CRC ^= (0x12 << 24)`

Następnie wykonywane jest **8 iteracji bitowych**:

```
if (CRC & 0x80000000)
    CRC = (CRC << 1) ^ 0x04C11DB7;
else
    CRC = CRC << 1;
```

Po 8 iteracjach pierwszy bajt zostaje uwzględniony w CRC.

3. Przetwarzanie drugiego bajtu (0x34)

```
CRC ^= (0x34 << 24)
```

Ponownie wykonywane jest **8 iteracji bitowych** zgodnie z algorytmem.

4. Wynik końcowy

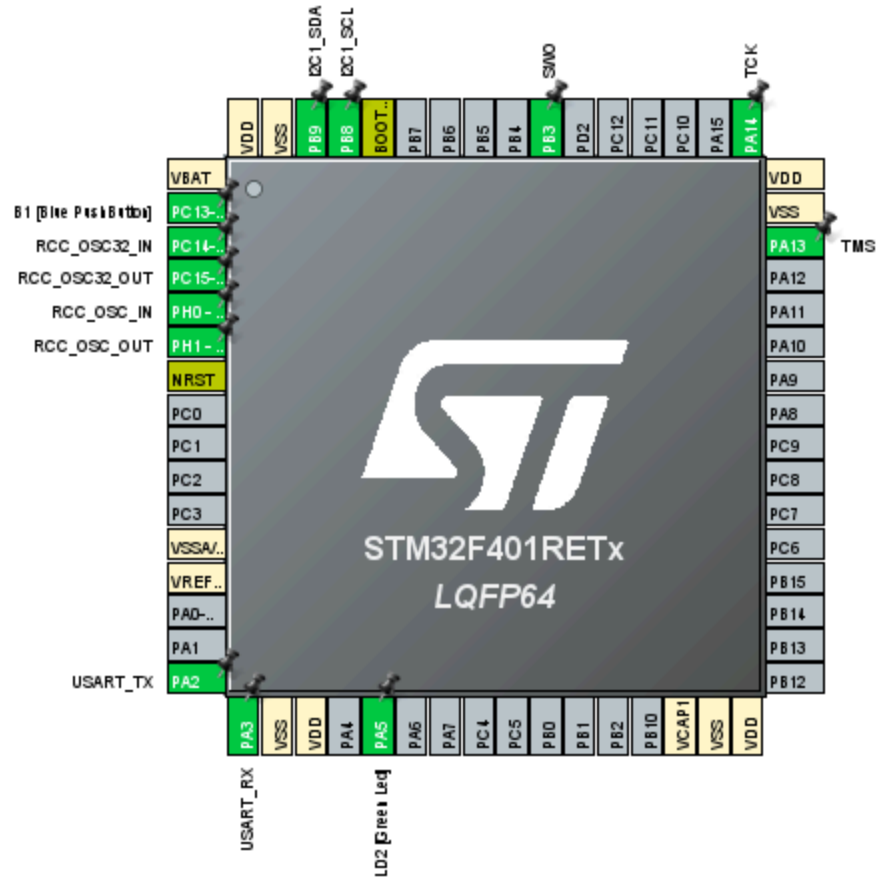
Po przetworzeniu wszystkich bajtów danych:

- **nie wykonuje się końcowego XOR**,
- zawartość rejestru CRC stanowi **wynik CRC-32**.

`CRC = wynik końcowy`

Wartość CRC przesyłana jest w ramce jako **4 bajty w konwencji Little-Endian**, zgodnie z przyjętą w projekcie reprezentacją danych.

8. Konfiguracja pinow i licznikow



8.1 Tabela wyprowadzeń (Pinout)

Poniższa tabela szczegółowo opisuje rolę każdego wykorzystanego pinu w projekcie:

Pin	Nazwa funkcji	Tryb pracy	Opis
PA2	**USART2_TX**	Alternate Function	Transmisja danych do PC (UART)
PA3	**USART2_RX**	Alternate Function	Odbiór komend z PC (UART)
PA5	**GPIO_Output**	Push-Pull	Sterowanie diodą LD2 (Status systemu)
PB8	**I2C1_SCL**	Alternate Function (Open-Drain)	Linia zegarowa magistrali I2C
PB9	**I2C1_SDA**	Alternate Function (Open-Drain)	Linia danych magistrali I2C
NRST	**Reset**	Input	Fizyczny przycisk restartu układu

8.2 Konfiguracja interfejsów komunikacyjnych

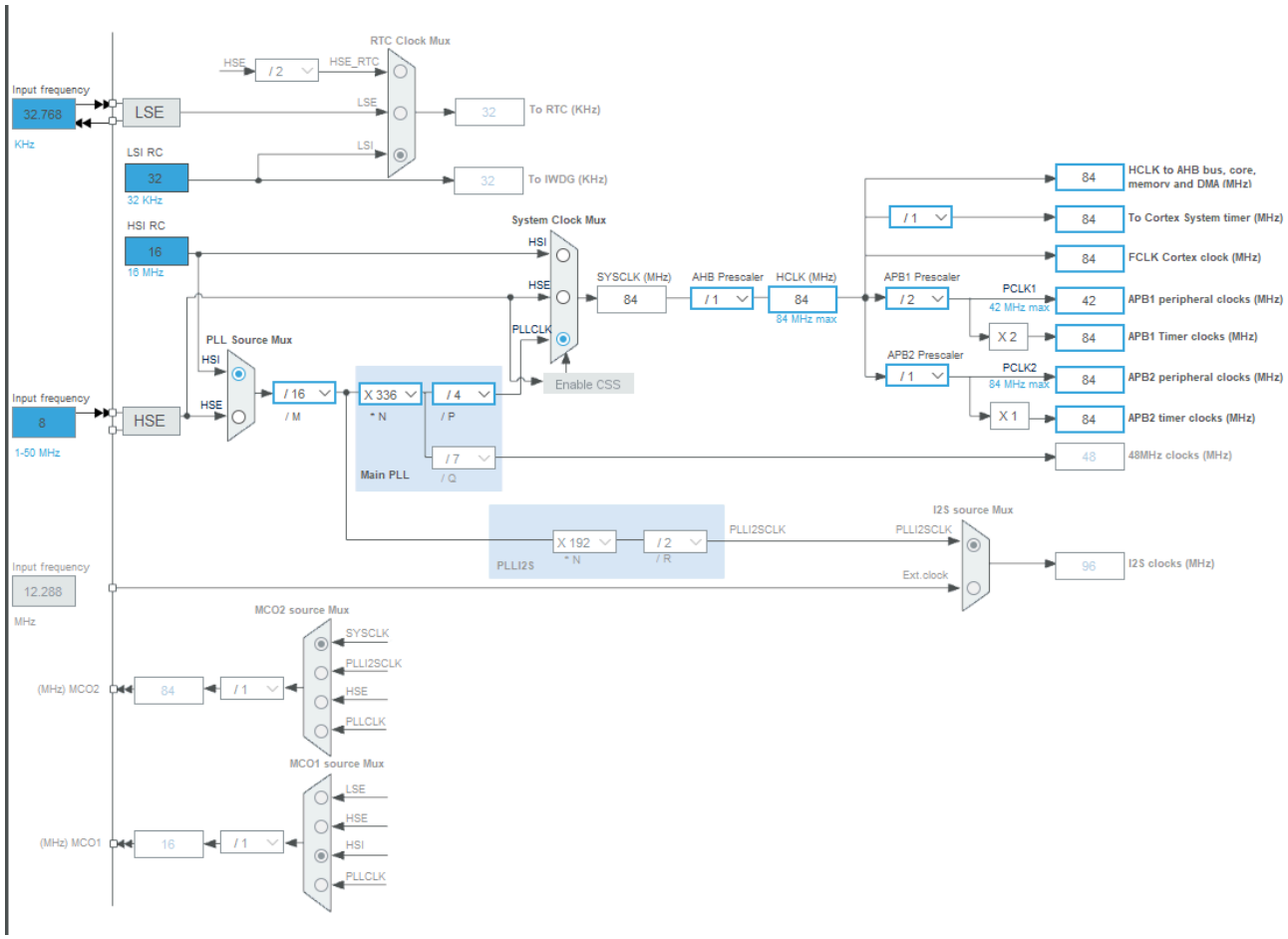
1. **I2C1 (Sensor MPU6050):**
   - Interfejs skonfigurowany w trybie **Standard Mode** z częstotliwością taktowania **100 kHz**.
   - Wykorzystano piny **PB8** oraz **PB9** z włączonymi wewnętrznymi rezystorami podciągającymi (**Pull-up**).
   - Do obsługi odczytu danych wykorzystano kontroler **DMA1 (Stream 0)**, co pozwala na nieblokujące pobieranie próbek pomiarowych.
2. **USART2 (Komunikacja z PC):**
   - Prędkość transmisji: **115200 baud**.
   - Format danych: **8 bitów danych, 1 bit stopu, brak parzystości**.
   - Obsługa odbioru realizowana jest poprzez przerwania (**Interrupt**), co umożliwia natychmiastowe parsowanie przychodzących bajtów przez maszynę stanów.

8.3 Konfiguracja Timera systemowego

Do odmierzania precyzyjnych interwałów próbkowania (domyślnie co **10 ms**) wykorzystano timer sprzętowy **TIM3**:

- **Prescaler:** Ustawiony tak, aby częstotliwość wejściowa licznika wynosiła **10 kHz**.
- **Counter Period (ARR):** Skonfigurowany na wartość generującą przerwanie z częstotliwością **100 Hz**.

- W przerwaniu timera wyzwalana jest funkcja startująca odczyt DMA z akcelerometru.



8.4 Konfiguracja interfejsu USART2 (Komunikacja z PC)

Interfejs USART2 służy jako główny kanał komunikacyjny projektu, pracując w trybie asynchronicznym.

- **Parametry transmisji:** Prędkość została ustalona na **115200 baud**, co zapewnia kompromis między szybkością a niskim poziomem błędów przy transmisji przez wbudowany programator ST-Link.
- **Tryb odbioru:** Wykorzystano **przerwania (Interrupt Mode)**. Każdy odebrany bajt wywołuje funkcję `HAL_UART_RxCpltCallback`, która natychmiast umieszcza go w autorskim buforze kołowym. Pozwala to na uniknięcie blokowania procesora podczas oczekiwania na dane z komputera.
- **Tryb nadawania:** Realizowany w pełni asynchronicznie za pomocą przerwań (`HAL_UART_Transmit_IT`). Zaimplementowano autorski system powiadamiania sprzętu (tzw. funkcja *tx_kick*), który pobiera bajty z bufora kołowego TX i wysyła je w świat na przerwaniach, zachowując płynność i integralność protokołu.

8.5 Konfiguracja magistrali I2C1 z obsługą DMA

Komunikacja z czujnikiem MPU6050 została zoptymalizowana tak, aby zminimalizować obciążenie jednostki CPU.

- **Parametry I2C:** Magistrala pracuje w trybie **Standard Mode (100 kHz)**. Wybór tej prędkości podyktowany był chęcią uzyskania maksymalnej stabilności przy połączeniach realizowanych za pomocą przewodów stykowych.
- **Wykorzystanie DMA (Direct Memory Access):**
  - Zastosowano **DMA1 Stream 0 Channel 1** do obsługi odczytu danych z rejestrów akcelerometru.
  - **Zasada działania:** Procesor jedynie inicjuje żądanie odczytu 6 bajtów danych (`HAL_I2C_Mem_Read_DMA`). Sam transfer danych z czujnika do pamięci RAM mikrokontrolera odbywa się całkowicie w tle, bez udziału CPU.
  - **Synchronizacja:** Po zakończeniu transferu DMA, generowane jest przerwanie, które wywołuje callback `HAL_I2C_MemRxCpltCallback`, gdzie następuje przeliczenie surowych danych na format użyteczny dla aplikacji.

8.6 Mechanizm buforowania (Ring Buffer)

Aby zapewnić płynność działania i uniknąć utraty danych przy szybkim próbkowaniu, w projekcie zaimplementowano dwa bufory cykliczne:

1. **Bufor UART RX:** Chroni przed przepełnieniem podczas odbierania długich ramek konfiguracyjnych (np. ustawianie czasu).
2. **Bufor próbek pomiarowych:** O rozmiarze 1024 próbek, pozwalający na przechowywanie historii ruchu, którą można pobrać komendą `CMD_GET_ARCHIVE`.